INFORMATICA: CHEATCODE DEFINITIVO PER INGEGNERIA INFORMATICA

Guida pratica con schemi rapidi, tutorial facili e decision tree per programmare senza impazzire

---

COME USARE QUESTO EBOOK

Informatica = **LOGICA** + **SINTASSI** + **PROBLEM SOLVING**. Il problema non è la complessità, è capire COME PENSARE come un programmatore.

Problema tipico dello studente di Ingegneria Informatica:

- Troppi linguaggi (C, Java, Python... quale usare?)
- "Compila ma non fa quello che voglio"
- Algoritmi che sembrano magia nera
- Strutture dati confuse (array, liste, alberi, grafe...)
- "Cosa sono puntatori? Perché segmentation fault?"

La soluzione NON è memorizzare sintassi. È capire LOGICA + avere SCHEMI RAPIDI.

Questo eBook fornisce:

- ✅ Tutorial pratici step-by-step (zero teoria inutile)
- ✅ Decision tree per scegliere struttura dati giusta
- ✅ Schemi rapidi per algoritmi comuni
- ✅ Debug workflow (come trovare errori velocemente)
- ✅ Pattern di codice riutilizzabili
- ✅ Trucchi per compilare/eseguire senza impazzire

Obiettivo: Capire la LOGICA della programmazione + scrivere codice funzionante velocemente.

🎯 STRATEGIA GENERALE - DAL CAOS AL CODICE

Il Grande Problema dell'Informatica

Troppi concetti che sembrano astratti:

- Variabili, puntatori, riferimenti
- Cicli for vs while vs do-while
- Array vs liste vs stack vs code
- Ricorsione vs iterazione
- Complessità O(n) vs O(log n) vs O(n^2)

La chiave: PENSARE PRIMA DI SCRIVERE + SCHEMI RAPIDI

Ogni concetto ha:

1. **Interpretazione pratica** (cosa serve DAVVERO)
2. **Schema codice** (template riutilizzabile)
3. **Quando usarlo** (decision tree)
4. **Errori comuni** (e come evitarli)

Metodo di Studio "CAPIRE → COPIARE → MODIFICARE → CREARE"

FASE 1: CAPIRE (prima di scrivere codice)

- Leggi problema 3 volte
- Disegna su carta: input → elaborazione → output
- Identifica pattern (è un ciclo? Una ricerca? Un ordinamento?)
- Guarda esempio pratico con numeri reali

FASE 2: COPIARE (pratica guidata)

- Usa template forniti
- Copia codice funzionante
- Compila ed esegui (verifica che funziona)
- Cambia valori input per testare

FASE 3: MODIFICARE (adattare)

- Modifica parte del codice
- Aggiungi printf/console.log per vedere cosa succede
- Sperimenta con variazioni

FASE 4: CREARE (autonomia)

- Scrivi codice da zero
- Segui decision tree
- Usa schemi memorizzati

Non saltare Fase 1. Carta e penna prima, codice dopo.

□ PARTE 1: FONDAMENTI - PENSARE COME UN COMPUTER

Variabili - CONTENITORI DI DATI

Interpretazione pratica: Variabile = scatola con etichetta (nome) che contiene un valore.

Cosa serve specificare:

1. **Tipo** (int, float, char, string...)
2. **Nome** (identificatore univoco)
3. **Valore** (opzionale, altrimenti valore di default)

Esempio C:
```
int età = 25; // intero
float altezza = 1.75; // decimale
char iniziale = 'E'; // singolo carattere
```

Esempio Java:
```
int età = 25;
double altezza = 1.75;
String nome = "Edoardo";
```

Esempio Python:
```
età = 25 # tipo inferito automaticamente
altezza = 1.75
nome = "Edoardo"
```

 REGOLA D'ORO: Nomi variabili devono essere DESCRITTIVI. No `x`, `y`, `temp`. Sì `età`, `somma_totale`, `numero_studenti`.

Tipi di Dati - SCHEMA RAPIDO

Tipo	Cosa contiene	Esempi	Dimensione
int	Numeri interi	-5, 0, 42, 1000	4 byte (tipico)
float/double	Decimali	3.14, -0.5, 2.718	4/8 byte
char	Singolo carattere	'a', '7', '$'	1 byte
string	Testo	"Ciao", "Test123"	Variabile
bool	Vero/Falso	true, false	1 byte

 DECISION TREE tipi:

```
Che tipo di dato?
↓
Numero?
├─ SÌ → Ha decimali?
│  ├─ SÌ → float/double
│  └─ NO → int
│
├─ Testo?
│  ├─ Singolo carattere → char
│  └─ Più caratteri → string
│
└─ Vero/Falso → bool
```

Operatori - COSA PUOI FARE CON I DATI

Operatori aritmetici:

- addizione
- sottrazione
- moltiplicazione
  / divisione
  % modulo (resto divisione)

Attenzione divisione interi:
int a = 5 / 2; // risultato = 2 (NON 2.5!)
float b = 5.0 / 2; // risultato = 2.5

Operatori di confronto:
== uguale
!= diverso
< minore

> maggiore
> <= minore o uguale
> = maggiore o uguale

Operatori logici:
&& AND (entrambi veri)
|| OR (almeno uno vero)
! NOT (negazione)

 ERRORE COMUNE: Confondere `=` (assegnazione) con `==` (confronto)

if (x = 5) // ✗ SBAGLIATO (assegna 5 a x)
if (x == 5) // ✓ CORRETTO (confronta x con 5)

PARTE 2: STRUTTURE DI CONTROLLO - DECIDERE E RIPETERE

If-Else - PRENDERE DECISIONI

Schema base:
if (condizione) {
// esegui se condizione vera
} else {
// esegui se condizione falsa
}

If-else if-else (decisioni multiple):
if (voto >= 90) {
printf("Eccellente\n");
} else if (voto >= 75) {
printf("Buono\n");

```
} else if (voto >= 60) {
printf("Sufficiente\n");
} else {
printf("Insufficiente\n");
}
```

 BEST PRACTICE:

- Condizione più specifica PRIMA
- Usa graffe {} anche per singola istruzione (evita errori)
- Indenta correttamente (leggibilità)

Switch - SELEZIONE MULTIPLA

Quando usare: Confronto singola variabile con valori fissi.

Schema:

```
switch (variabile) {
case valore1:
// codice
break;
case valore2:
// codice
break;
default:
// codice se nessun case
}
```

Esempio:

```
switch (giorno) {
case 1:
printf("Lunedì\n");
break;
case 2:
printf("Martedì\n");
break;
// ...
default:
printf("Giorno non valido\n");
}
```

 NON DIMENTICARE `break`! Altrimenti esegue anche case successivi.

Ciclo For - RIPETERE N VOLTE

Schema:
```
for (inizializzazione; condizione; incremento) {
// codice da ripetere
}
```

Esempio classico (0 a 9):
```
for (int i = 0; i < 10; i++) {
printf("%d\n", i);
}
```

Anatomia del for:

1. **Inizializzazione:** `int i = 0` (eseguita UNA volta)
2. **Condizione:** `i < 10` (controllata PRIMA di ogni iterazione)
3. **Incremento:** `i++` (eseguito DOPO ogni iterazione)
4. **Body:** codice dentro graffe

 PATTERN COMUNI:

Contare da 1 a N:
```
for (int i = 1; i <= N; i++)
```

Contare all'indietro:
```
for (int i = N; i > 0; i--)
```

Passo 2:
```
for (int i = 0; i < 10; i += 2) // 0, 2, 4, 6, 8
```

Iterare array:
```
for (int i = 0; i < lunghezza; i++) {
printf("%d\n", array[i]);
}
```

Ciclo While - RIPETERE FINCHÉ...

Schema:
```
while (condizione) {
// codice da ripetere
}
```

Quando usare: Numero iterazioni NON noto a priori.

Esempio - input utente:
```
int numero;
printf("Indovina numero (1-10): ");
scanf("%d", &numero);

while (numero != 7) {
printf("Sbagliato! Riprova: ");
scanf("%d", &numero);
}
printf("Corretto!\n");
```

 ATTENZIONE: Verifica che condizione diventi falsa, altrimenti **loop infinito**!

Loop infinito (errore comune):
```
int i = 0;
while (i < 10) {
printf("%d\n", i);
// dimenticato i++! Loop infinito
}
```

Do-While - ESEGUI ALMENO UNA VOLTA

Schema:
```
do {
// codice da ripetere
} while (condizione);
```

Differenza con while: Body eseguito ALMENO UNA VOLTA (condizione controllata DOPO).

Esempio - menu:
int scelta;
do {
printf("1. Opzione A\n");
printf("2. Opzione B\n");
printf("0. Esci\n");
scanf("%d", &scelta);

```
// elabora scelta...
```

} while (scelta != 0);

Decision Tree - Quale Ciclo Usare?

Devo ripetere codice?
↓
Quante volte?
├─ So ESATTAMENTE (es. 10 volte, lunghezza array)
│ → FOR
│
├─ Dipende da CONDIZIONE ma voglio eseguire ALMENO UNA VOLTA
│ → DO-WHILE
│
└─ Dipende da CONDIZIONE
→ WHILE

 REGOLA PRATICA: 90% casi → FOR. Resto → WHILE.

PARTE 3: ARRAY E STRUTTURE DATI - ORGANIZZARE I DATI

Array - LISTA DI ELEMENTI OMOGENEI

Interpretazione: Array = scaffale con N caselle numerate (indici 0, 1, 2...)

Dichiarazione C:
int numeri[5]; // array 5 interi (non inizializzato)
int valori[5] = {10, 20, 30, 40, 50}; // con valori iniziali

Accesso elementi:
printf("%d\n", valori[0]); // primo elemento (10)
valori[2] = 99; // modifica terzo elemento

 ATTENZIONE: Indici partono da 0!

- Array di 5 elementi: indici 0, 1, 2, 3, 4
- Accedere a `valori[5]` = **ERRORE** (fuori bounds)

Array - Pattern Comuni

Inizializzare tutti elementi a 0:
int array[100] = {0}; // tutti zeri

Riempire array da input:
int n = 5;
int array[5];
for (int i = 0; i < n; i++) {
printf("Elemento %d: ", i);
scanf("%d", &array[i]);
}

Somma elementi:
int somma = 0;
for (int i = 0; i < n; i++) {
somma += array[i];
}

Trovare massimo:
int max = array[0];
for (int i = 1; i < n; i++) {
if (array[i] > max) {
max = array[i];
}
}

Stampare array:

```
for (int i = 0; i < n; i++) {
printf("%d ", array[i]);
}
printf("\n");
```

Stringhe - ARRAY DI CARATTERI

In C: Stringa = array di `char` terminato da `\0` (null terminator)

```
char nome[10] = "Edoardo"; // 'E','d','o','a','r','d','o','\0'
```

 DIMENSIONE: Serve 1 carattere in più per `\0`!

Funzioni stringhe (include `<string.h>`):

```
strlen(str) // lunghezza
strcpy(dest, src) // copia
strcat(dest, src) // concatena
strcmp(str1, str2) // confronta (0 se uguali)
```

Esempio:

```
char s1[20] = "Ciao";
char s2[20] = "Mondo";

strcat(s1, " ");
strcat(s1, s2); // s1 = "Ciao Mondo"

if (strcmp(s1, "Ciao Mondo") == 0) {
printf("Uguali\n");
}
```

Matrici - ARRAY BIDIMENSIONALI

Interpretazione: Matrice = tabella con righe e colonne

Dichiarazione:

```
int matrice[3][4]; // 3 righe, 4 colonne
```

Inizializzazione:

```
int mat[3][3] = {
{1, 2, 3},
{4, 5, 6},
```

```
{7, 8, 9}
};
```

Accesso elementi:

```
int valore = mat[1][2]; // riga 1, colonna 2 → 6
mat[0][0] = 99; // modifica primo elemento
```

Iterare matrice (doppio ciclo):

```
for (int i = 0; i < righe; i++) {
for (int j = 0; j < colonne; j++) {
printf("%d ", mat[i][j]);
}
printf("\n");
}
```

Liste - STRUTTURE DINAMICHE

Problema array: Dimensione FISSA (decisa a compile-time)

Soluzione: Liste dinamiche (linked list)

Nodo lista (C):

```
struct Nodo {
int dato;
struct Nodo* prossimo;
};
```

Operazioni base:

1. Inserimento (testa, coda, posizione)
2. Cancellazione
3. Ricerca
4. Attraversamento

 PRO/CONTRO:

Array:

- ✅ Accesso rapido O(1) per indice
- ❌ Dimensione fissa
- ❌ Inserimento/cancellazione lento

Liste:

- ✅ Dimensione dinamica
- ✅ Inserimento/cancellazione rapido
- ❌ Accesso sequenziale O(n)

Decision Tree - Quale Struttura Dati?

```
Devo memorizzare dati?
↓
Quanti elementi?
├─ NUMERO FISSO noto a priori
│  → ARRAY
│
└─ NUMERO VARIABILE o molto grande
↓
Tipo accesso?
├─ Per INDICE (accesso random)
│  → Vector/ArrayList (array dinamico)
│
├─ Solo INIZIO/FINE (LIFO o FIFO)
│  ├─ LIFO (ultimo entrato, primo uscito) → STACK
│  └─ FIFO (primo entrato, primo uscito) → QUEUE
│
└─ Ricerche FREQUENTI
→ Hash Table / Set
```

🔧 PARTE 4: FUNZIONI - RIUSARE CODICE

Funzioni - MODULARIZZARE

Interpretazione: Funzione = "mini-programma" che fa una cosa specifica.

Vantaggi:

- ✅ Riusabilità (scrivi una volta, usa molte volte)
- ✅ Leggibilità (codice più chiaro)
- ✅ Manutenibilità (modifichi in un solo posto)
- ✅ Testing (testi singola funzione)

Anatomia funzione:

```
tipo_ritorno nome_funzione(parametri) {
// corpo funzione
return valore;
}
```

Esempio:

```
int somma(int a, int b) {
return a + b;
}

int main() {
int risultato = somma(5, 3); // 8
printf("%d\n", risultato);
return 0;
}
```

Funzioni - Pattern Comuni

Funzione void (non ritorna valore):

```
void stampa_messaggio(char* messaggio) {
printf("%s\n", messaggio);
}
```

Funzione con array:

```
int somma_array(int arr[], int n) {
int somma = 0;
for (int i = 0; i < n; i++) {
somma += arr[i];
}
return somma;
}
```

Funzione bool (controllo):

```
bool è_pari(int n) {
return (n % 2 == 0);
}
```

Funzione con parametri default (C++):

```
int potenza(int base, int esponente = 2) {
int risultato = 1;
```

```
for (int i = 0; i < esponente; i++) {
risultato *= base;
}
return risultato;
}

// potenza(5) → 25 (esponente default = 2)
// potenza(5, 3) → 125
```

Ricorsione - FUNZIONE CHE CHIAMA SE STESSA

Interpretazione: Ricorsione = problema risolto in termini di versione più piccola dello stesso problema.

Componenti essenziali:

1. **Caso base** (condizione di stop)
2. **Chiamata ricorsiva** (verso caso base)

Esempio - Fattoriale:

```
int fattoriale(int n) {
// CASO BASE
if (n == 0 || n == 1) {
return 1;
}
// CHIAMATA RICORSIVA
return n * fattoriale(n - 1);
}

// fattoriale(5) = 5 * fattoriale(4)
// = 5 * 4 * fattoriale(3)
// = 5 * 4 * 3 * fattoriale(2)
// = 5 * 4 * 3 * 2 * fattoriale(1)
// = 5 * 4 * 3 * 2 * 1 = 120
```

Esempio - Fibonacci:

```
int fibonacci(int n) {
if (n <= 1) return n;
return fibonacci(n-1) + fibonacci(n-2);
}
```

 ATTENZIONE: Ricorsione elegante ma può essere inefficiente (stack overflow se troppo profonda).

Ricorsione vs Iterazione:

- **Ricorsione:** Codice più pulito, intuitivo per problemi ricorsivi (alberi, grafi)
- **Iterazione:** Più efficiente (meno overhead), nessun rischio stack overflow

---

PARTE 5: DEBUG - TROVARE E RISOLVERE ERRORI

Tipi di Errori

Tipo	Quando	Come risolvere
Sintassi	Compilazione	Leggi messaggio compilatore
Runtime	Esecuzione	Debugger, print
Logici	Output sbagliato	Tracciare variabili

Errori Sintassi - COMPILE-TIME

Esempi comuni:
int x = 5 // ✗ manca ;

if (x = 5) { } // ✗ = invece di ==

printf("Ciao" // ✗ manca)

int array[5]
array[0] = 10; // ✗ manca ;

 STRATEGIA:

1. Leggi messaggio errore (indica riga)
2. Controlla riga indicata + precedente
3. Cerca: ; mancante, parentesi non chiuse, virgolette non chiuse

---

Errori Runtime - ESECUZIONE

Esempi comuni:

Divisione per zero:
int a = 5 / 0; // ✗ crash

Array out of bounds:
int arr[5];
arr[10] = 99; // ✗ accesso fuori limiti

Segmentation fault (accesso memoria invalida):
int* ptr = NULL;
*ptr = 5; // ✗ dereferenzia puntatore null

 STRATEGIA:

1. Identifica riga crash (debugger o ultimo print)
2. Controlla indici array, puntatori null
3. Aggiungi print PRIMA della riga problematica

Debug con Printf - TECNICA UNIVERSALE

Strategia print strategici:

```
printf("DEBUG: Entrato funzione X\n");
printf("DEBUG: valore x = %d\n", x);
printf("DEBUG: Prima del ciclo\n");
for (...) {
printf("DEBUG: i = %d, arr[i] = %d\n", i, arr[i]);
}
printf("DEBUG: Dopo il ciclo\n");
```

 BEST PRACTICE:

- Prefisso "DEBUG:" (facile trovare e rimuovere dopo)
- Stampa VALORE variabili (non solo "qui")
- Stampa PRIMA e DOPO sezioni critiche

Errori Logici - OUTPUT SBAGLIATO

Esempio:
```
// Voglio somma 1 a N
int somma = 0;
for (int i = 1; i < N; i++) { // ✗ dovrebbe essere i <= N
somma += i;
}
```

 STRATEGIA:

1. Traccia esecuzione su carta (valori piccoli)
2. Verifica condizioni cicli (`<` vs `<=`)
3. Controlla casi limite (N=0, N=1, array vuoto)

Rubber Duck Debugging: Spiega codice a qualcuno (o a papera di gomma). Spesso trovi errore spiegando.

PARTE 6: ALGORITMI FONDAMENTALI - PROBLEMI COMUNI

Algoritmo - RICETTA PER RISOLVERE PROBLEMA

Algoritmo = sequenza finita di passi per risolvere problema

Caratteristiche:

- Input definito
- Output definito
- Passi non ambigui
- Finito (termina)
- Efficace (risolve problema)

Ricerca Lineare - TROVARE ELEMENTO

Problema: Dato array, trovare se elemento presente.

Algoritmo:
```
int ricerca_lineare(int arr[], int n, int target) {
for (int i = 0; i < n; i++) {
if (arr[i] == target) {
```

return i; // trovato, ritorna indice
}
}
return -1; // non trovato
}

Complessità: O(n) - nel caso peggiore controlli tutti elementi

---

Ricerca Binaria - RICERCA VELOCE (ARRAY ORDINATO)

Prerequisito: Array DEVE essere ordinato

Algoritmo: Dividi e conquista

1. Confronta target con elemento centrale
2. Se uguale → trovato
3. Se target < centrale → cerca metà sinistra
4. Se target > centrale → cerca metà destra

Codice:
int ricerca_binaria(int arr[], int n, int target) {
int sinistra = 0;
int destra = n - 1;

```
while (sinistra <= destra) {
    int centro = (sinistra + destra) / 2;

    if (arr[centro] == target) {
        return centro;  // trovato
    }
    if (arr[centro] < target) {
        sinistra = centro + 1;  // cerca destra
    } else {
        destra = centro - 1;    // cerca sinistra
    }
}
return -1;  // non trovato
```

}

Complessità: O(log n) - MOLTO più veloce di lineare

Esempio: Array 1000 elementi

- Ricerca lineare: fino a 1000 confronti
- Ricerca binaria: massimo 10 confronti ($\log_2 1000 \approx 10$)

Ordinamento - METTERE IN ORDINE

Bubble Sort - SEMPLICE MA LENTO

Idea: Confronta coppie adiacenti, scambia se in ordine sbagliato. Ripeti finché ordinato.

```
void bubble_sort(int arr[], int n) {
for (int i = 0; i < n-1; i++) {
for (int j = 0; j < n-i-1; j++) {
if (arr[j] > arr[j+1]) {
// scambia
int temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;
}
}
}
}
```

Complessità: $O(n^2)$ - lento per array grandi

Selection Sort - TROVA MINIMO

Idea: Trova elemento minimo, mettilo in posizione corretta. Ripeti per resto array.

```
void selection_sort(int arr[], int n) {
for (int i = 0; i < n-1; i++) {
int min_idx = i;
for (int j = i+1; j < n; j++) {
if (arr[j] < arr[min_idx]) {
min_idx = j;
}
```

```
}
// scambia elemento minimo con posizione i
int temp = arr[i];
arr[i] = arr[min_idx];
arr[min_idx] = temp;
}
}
```

Complessità: $O(n^2)$

Quick Sort - VELOCE (PIÙ USATO)

Idea: Scegli pivot, metti elementi < pivot a sinistra, > pivot a destra. Ricorsivamente ordina sottosezioni.

Complessità: O(n log n) media - MOLTO più veloce

 PRATICA: Usa funzioni libreria `qsort()` (C), `Arrays.sort()` (Java), `sorted()` (Python)

Complessità Algoritmi - QUANTO È VELOCE?

Notazione O grande (Big O): Misura crescita tempo esecuzione al crescere input.

Complessità	Nome	Esempio
O(1)	Costante	Accesso array per indice
O(log n)	Logaritmica	Ricerca binaria
O(n)	Lineare	Ricerca lineare, somma array
O(n log n)	Linearitmica	Quick sort, merge sort
$O(n^2)$	Quadratica	Bubble sort, doppio ciclo
$O(2^n)$	Esponenziale	Fibonacci ricorsivo

 REGOLA: Preferisci sempre complessità minore (O(log n) > O(n) > $O(n^2)$)

🔷 PARTE 7: LINGUAGGI - COMPARAZIONE RAPIDA

C vs Java vs Python - SCHEMA RAPIDO

Feature	C	Java	Python
Tipizzazione	Statica	Statica	Dinamica
Compilato	Sì	Bytecode	Interpretato
Gestione memoria	Manuale	Automatica (GC)	Automatica (GC)
Velocità	⚡⚡⚡	⚡⚡	⚡
Sintassi	Complessa	Media	Semplice
Uso	OS, embedded	Enterprise, Android	Scripting, AI, web

Hello World - 3 Linguaggi

C:

```
#include <stdio.h>

int main() {
printf("Hello World\n");
return 0;
}
```

Java:

```
public class Main {
public static void main(String[] args) {
System.out.println("Hello World");
}
}
```

Python:

```
print("Hello World")
```

Quando Usare Quale Linguaggio?

Usa C quando:

- ✅ Massima performance (sistemi operativi, driver)
- ✅ Controllo hardware diretto
- ✅ Embedded systems
- ✅ Il corso lo richiede 😄

Usa Java quando:

- ✅ Applicazioni enterprise
- ✅ App Android
- ✅ Vuoi portabilità (JVM)
- ✅ Progetti grandi con team

Usa Python quando:

- ✅ Prototipazione rapida
- ✅ Data science, AI, machine learning
- ✅ Scripting e automazione
- ✅ Vuoi scrivere codice velocemente

□ PARTE 8: SETUP E COMPILAZIONE - FAR GIRARE IL CODICE

Compilare in C - GCC

Comando base:
gcc programma.c -o programma
./programma

Con warning (SEMPRE consigliato):
gcc -Wall -Wextra programma.c -o programma

Con debug info:
gcc -g programma.c -o programma
gdb programma

Compilare in Java

Compilazione:
javac Programma.java

Esecuzione:
java Programma

Eseguire Python

Diretto:
python3 script.py

Modalità interattiva (REPL):
python3

```
print("Test")
exit()
```

CHECKLIST PRIMA ESAME

Sintassi Base

- [ ] Dichiarare variabili (int, float, char, string)
- [ ] Operatori aritmetici e logici
- [ ] Printf/scanf (C) o equivalenti
- [ ] If-else, switch
- [ ] For, while, do-while

Array e Stringhe

- [ ] Dichiarare e inizializzare array
- [ ] Accedere elementi (indici da 0!)
- [ ] Iterare array con for
- [ ] Stringhe (array di char con \0)
- [ ] Matrici (array 2D)

Funzioni

- [ ] Definire funzione con parametri
- [ ] Return value
- [ ] Void functions
- [ ] Passaggio array a funzioni

Algoritmi Comuni

- [ ] Somma elementi array
- [ ] Trovare massimo/minimo
- [ ] Ricerca lineare
- [ ] Ordinamento (almeno bubble sort)
- [ ] Calcolare media

Puntatori (se C)

- [ ] Dichiarare puntatore: `int* ptr;`
- [ ] Assegnare indirizzo: `ptr = &variabile;`
- [ ] Dereferenziare: `*ptr`
- [ ] Puntatori e array (equivalenza)

ERRORI DA EVITARE ALL'ESAME

1. **Dimenticare `;` alla fine istruzioni (C/Java)**
2. \textbf{Confondere `=` con `==`}: Assegnazione vs confronto
3. **Array out of bounds**: Indici da 0 a n-1, non 1 a n!
4. **Loop infinito**: Verifica condizione uscita
5. **Variabili non inizializzate**: Valori random!
6. **Dimenticare \textbackslash 0 nelle stringhe (C)**
7. **Divisione intera**: `5/2 = 2`, non 2.5
8. **Modificare indice ciclo dentro ciclo**: Comportamento imprevedibile
9. **Puntatori non inizializzati (C)**: Segmentation fault garantito
10. **Non testare casi limite**: Array vuoto, n=0, n=1

PIANO STUDIO 4 SETTIMANE

Settimana 1 - Basi:

- Giorni 1-2: Variabili, tipi, operatori, I/O
- Giorni 3-4: If-else, switch
- Giorni 5-6: Cicli (for, while)
- Giorno 7: 20 esercizi base (stampe, somme, cicli semplici)

Settimana 2 - Array e Funzioni:

- Giorni 1-2: Array, iterazione, operazioni base
- Giorni 3-4: Stringhe (se C)
- Giorni 5-6: Funzioni, parametri, return
- Giorno 7: 15 esercizi array e funzioni

Settimana 3 - Algoritmi:

- Giorni 1-2: Ricerca (lineare, binaria)
- Giorni 3-4: Ordinamento (bubble, selection)
- Giorni 5-6: Ricorsione (fattoriale, fibonacci)
- Giorno 7: 10 esercizi algoritmi

Settimana 4 - Puntatori e Ripasso:

- Giorni 1-2: Puntatori (se C), strutture dati avanzate
- Giorni 3-4: Matrici, problemi complessi
- Giorno 5: Simulazione esame completa
- Giorno 6: Rivedere errori, formulario
- Giorno 7: Ripasso veloce schemi

Giorno esame: Rileggi checklist + errori comuni

TRUCCHI MENTALI

Pensare Come Programmatore

1. Decomposizione: Problema grande = tanti problemi piccoli

"Scrivere programma gestione biblioteca"
↓

- Aggiungere libro
- Cercare libro
- Rimuovere libro
- Stampare lista

2. Pattern Recognition: "Ho già visto problema simile?"

3. Astrazione: Generalizzare soluzione

4. Testing: Prova con input semplici prima

Debug Mentale

Tecnica "Passo-passo":

1. Scrivi valori variabili su carta
2. Esegui mentalmente riga per riga
3. Aggiorna valori dopo ogni istruzione
4. Confronta con output atteso

Esempio:
```
int x = 5;
int y = 3;
x = x + y; // x = 8
y = x - y; // y = 5
x = x - y; // x = 3
// x e y scambiati!
```

CONCLUSIONE - DA ZERO A PROGRAMMATORE

Informatica non è "difficile". È "diversa".

Richiede:

- **Logica** (pensare passo-passo)
- **Precisione** (punto e virgola mancante = errore)
- **Pratica** (100 esercizi > 10 ore teoria)
- **Pazienza** (debug fa parte del processo)

Le 5 chiavi del successo:

1. **CARTA PRIMA, CODICE DOPO**: Disegna logica prima di scrivere
2. **COPIA-MODIFICA-CREA**: Usa template, poi personalizza
3. **TESTA SEMPRE**: Input piccoli, casi limite, scenari strani
4. **DEBUG STRATEGICO**: Printf ovunque, traccia variabili
5. **PRATICA COSTANTE**: 30 min/giorno > 5 ore una volta

Non serve essere genio. Serve:

- Capire logica (decision tree, schemi)
- Riconoscere pattern (questo problema = ricerca/ordinamento/...)
- Applicare template (riusa codice funzionante)
- Debuggare metodicamente (non a caso)

Con questo eBook + 4 settimane pratica = ESAME SUPERATO.

Il codice che sembra magico diventerà chiaro. Gli errori diventeranno normali.

Buono studio! 🚀

FINE EBOOK CHEATCODE INFORMATICA

Versione 1.0 - Dalla confusione al codice funzionante